

Part 0:

SID4: 0686

PORT_BASE: 8686

PREFIX: s0686

SEED: 686

VERIFY_SEED: 260686

DOMAIN_ID: 6

Local Model: llama3.2:latest

Tagged Commit Hash: eabc88cefb12b31577aa9a72ab7619b5480b166

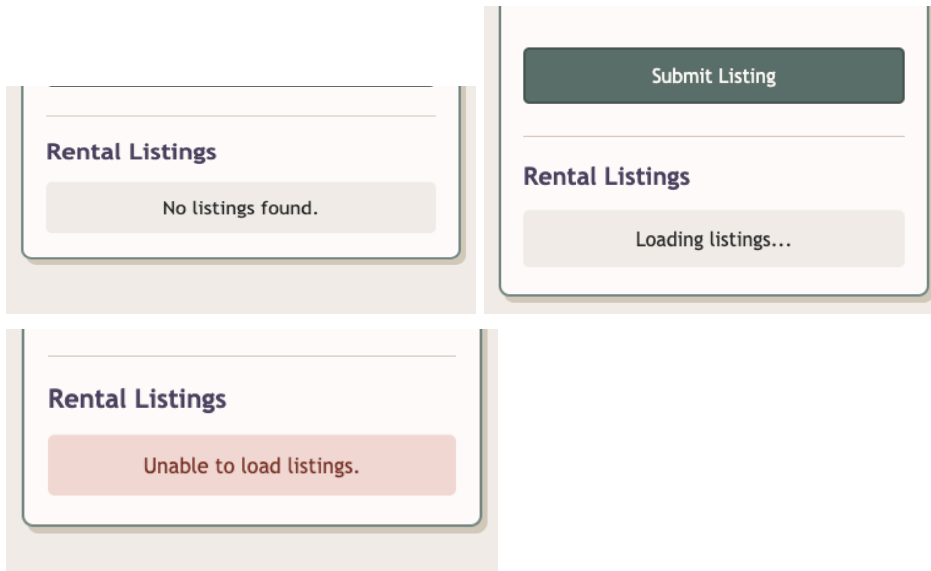
GitHub Repository: <https://github.com/farzaandalwai/data260-0686>

PART 1:

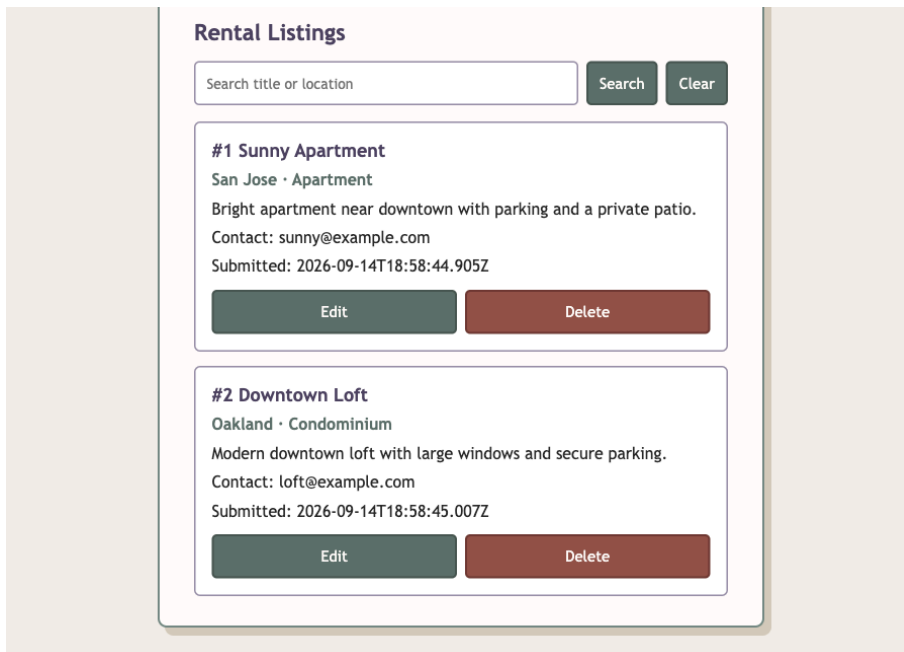
```
const showLoading = () => {  
  listingsList.innerHTML = "";  
  listingsStatus.hidden = false;  
  listingsStatus.classList.remove("error");  
  listingsStatus.textContent = "Loading listings...";  
};
```

```
const showEmpty = () => {  
  listingsList.innerHTML = "";  
  listingsStatus.hidden = false;  
  listingsStatus.classList.remove("error");  
  listingsStatus.textContent = "No listings found.";  
};
```

```
const showError = (message = "Unable to load listings.") => {  
  listingsList.innerHTML = "";  
  listingsStatus.hidden = false;  
  listingsStatus.classList.add("error");  
  listingsStatus.textContent = message;  
};
```



Part 2:



Rental Listings

Search title or location

Search

Clear

#1 Updated Sunny Home

Santa Clara · Apartment

Bright apartment near downtown with parking and a private patio.

Contact: sunny@example.com

Submitted: 2026-09-14T18:58:44.905Z

Edit

Delete

#2 Downtown Loft

Oakland · Condominium

Modern downtown loft with large windows and secure parking.

Contact: loft@example.com

Submitted: 2026-09-14T18:58:45.007Z

Edit

Delete

Rental Listings

updated sunny

Search

Clear

#1 Updated Sunny Home

Santa Clara · Apartment

Bright apartment near downtown with parking and a private patio.

Contact: sunny@example.com

Submitted: 2026-09-14T18:58:44.905Z

Edit

Delete

Rental Listings

Search

Clear

#2 Downtown Loft

Oakland · Condominium

Modern downtown loft with large windows and secure parking.

Contact: loft@example.com

Submitted: 2026-09-14T18:58:45.007Z

Edit

Delete

Rental Listings

Search

Clear

#1 Updated Sunny Home

Santa Clara · Apartment

Bright apartment near downtown with parking and a private patio.

Contact: sunny@example.com

Submitted: 2026-09-14T18:58:44.905Z

Edit

Delete

#2 Downtown Loft

Oakland · Condominium

Modern downtown loft with large windows and secure parking.

Contact: loft@example.com

Submitted: 2026-09-14T18:58:45.007Z

Edit

Delete

Rental Listings

Search

Clear

#1 Updated Sunny Home

Santa Clara · Apartment

Bright apartment near downtown with parking and a private patio.

Contact: sunny@example.com

Submitted: 2026-09-14T18:58:44.905Z

Edit

Delete

CREATE:

```

@app.post("/listings", response_model=RentalListing, status_code=201)
def create_listing(data: ListingInput):
    global next_id

    listing = RentalListing(
        id=next_id,
        propertyTitle=data.propertyTitle,
        location=data.location,
        submitterEmail=data.submitterEmail,
        description=data.description,
        propertyType=data.propertyType,
        termsAccepted=data.termsAccepted,
        submissionDate=data.submissionDate
        or datetime.now(timezone.utc).isoformat(),
    )

    listings.append(listing)
    next_id += 1
    return listing

```

UPDATE:

```

@app.put("/listings/{listing_id}", response_model=RentalListing)
def update_listing(listing_id: int, data: ListingInput):
    for index, listing in enumerate(listings):
        if listing.id == listing_id:
            updated_listing = RentalListing(
                id=listing_id,

```

```

        propertyTitle=data.propertyTitle,
        location=data.location,
        submitterEmail=data.submitterEmail,
        description=data.description,
        propertyType=data.propertyType,
        termsAccepted=data.termsAccepted,
        submissionDate=data.submissionDate or listing.submissionDate,
    )
    listings[index] = updated_listing
    return updated_listing

```

```

raise HTTPException(status_code=404, detail="Listing not found")

```

DELETE:

```

@app.delete("/listings/{listing_id}")
def delete_listing(listing_id: int):
    for index, listing in enumerate(listings):
        if listing.id == listing_id:
            listings.pop(index)
            return {"message": "Listing deleted", "id": listing_id}

    raise HTTPException(status_code=404, detail="Listing not found")

```

SEARCH:

```

@app.get("/listings", response_model=list[RentalListing])
def get_listings(search: str | None = None):
    if not search:
        return listings

    search_text = search.lower()
    return [
        listing
        for listing in listings
        if search_text in listing.propertyTitle.lower()
        or search_text in listing.location.lower()
    ]

```

PART 3:

AGENTSTATE:

```
class AgentState(TypedDict):
```

```
    title: str
    content: str
    email: str
    strict: bool
    task: str
    planner_proposal: dict
    reviewer_feedback: dict
    turn_count: int
```

Planner Reviewer:

```
def planner_node(state: AgentState) -> dict:
```

```
    print("---NODE: Planner---")
```

```
    result = model_client.complete(
        [{"role": "user", "content": prompt}],
        temperature=0.0,
        json_output=True,
    )
    proposal = json.loads(result["message"])
```

```
    return {
        "planner_proposal": proposal,
        "reviewer_feedback": {},
    }
```

```
def reviewer_node(state: AgentState) -> dict:
```

```
    print("---NODE: Reviewer---")
```

```
    result = model_client.complete(
        [{"role": "user", "content": prompt}],
        temperature=0.0,
        json_output=True,
    )
    feedback = json.loads(result["message"])
```

```
    return {"reviewer_feedback": feedback}
```

Supervisor and Router:

```

def supervisor_node(state: AgentState) -> dict:
    print("---NODE: Supervisor---")
    return {"turn_count": state["turn_count"] + 1}

def router_logic(state: AgentState):
    if not state["planner_proposal"]:
        return "planner"

    if not state["reviewer_feedback"]:
        return "reviewer"

    if state["reviewer_feedback"].get("changed", False):
        return "planner"

    return END

```

Graph Wiring:

```

workflow = StateGraph(AgentState)

workflow.add_node("supervisor", supervisor_node)
workflow.add_node("planner", planner_node)
workflow.add_node("reviewer", reviewer_node)

workflow.set_entry_point("supervisor")

workflow.add_conditional_edges(
    "supervisor",
    router_logic,
    {
        "planner": "planner",
        "reviewer": "reviewer",
        END: END,
    },
)

workflow.add_edge("planner", "supervisor")
workflow.add_edge("reviewer", "supervisor")

graph = workflow.compile()

```

Normal LangGraph execution ending after Reviewer approval.

```
$ python3.11 normal_graph_run
LANGGRAPH NORMAL RUN
---NODE: Supervisor---
{
  "supervisor": {
    "turn_count": 1
  }
}
---NODE: Planner---
{
  "planner": {
    "planner_proposal": {
      "tags": [
        "San Jose Rental",
        "3 Bedroom House",
        "Pool Access"
      ],
      "summary": "Spacious 3 bedroom house near downtown San Jose with garage parking and pool access"
    },
    "reviewer_feedback": {}
  }
}
---NODE: Supervisor---
{
  "supervisor": {
    "turn_count": 2
  }
}
---NODE: Reviewer---
{
  "reviewer": {
    "reviewer_feedback": {
      "tags": [
        "San Jose Rental",
        "3 Bedroom House",
        "Pool Access"
      ],
      "summary": "Spacious 3 bedroom house near downtown San Jose with garage parking and pool access",
      "changed": false,
      "reason": "Summary is accurate and within the 25-word limit"
    }
  }
}
---NODE: Supervisor---
{
  "supervisor": {
    "turn_count": 3
  }
}
---GRAPH COMPLETE---
```

Correction-loop execution where Reviewer detects an issue and routes back to Planner:

```

$ python3.11 correction_loop_run
LANGGRAPH CORRECTION LOOP RUN
---NODE: Supervisor---
{
  "supervisor": {
    "turn_count": 1
  }
}
---NODE: Planner---
{
  "planner": {
    "planner_proposal": {
      "tags": [
        "San Jose Rental",
        "3 Bedroom House",
        "Pool Access"
      ],
      "summary": "Spacious 3 bedroom house near downtown San Jose with garage parking and pool access"
    },
    "reviewer_feedback": {}
  }
}
---NODE: Supervisor---
{
  "supervisor": {
    "turn_count": 2
  }
}
---NODE: Reviewer---
---TEMP REVIEW RESULT: changed=true---
{
  "reviewer": {
    "reviewer_feedback": {
      "tags": [
        "San Jose Rental",
        "3 Bedroom House",
        "Pool Access"
      ],
      "summary": "Spacious 3 bedroom house near downtown San Jose with garage parking and pool access",
      "changed": true,
      "reason": "Improve the proposal and review it again."
    }
  }
}
---NODE: Supervisor---
{
  "supervisor": {
    "turn_count": 3
  }
}
---NODE: Planner---
{
  "planner": {
    "planner_proposal": {
      "tags": [
        "San Jose Rental",
        "3 Bedroom House",
        "Pool Access"
      ],
      "summary": "Spacious 3 bedroom house near downtown San Jose with garage parking and pool access"
    },
    "reviewer_feedback": {}
  }
}
---NODE: Supervisor---
{
  "supervisor": {
    "turn_count": 4
  }
}
---NODE: Reviewer---
---TEMP REVIEW RESULT: changed=false---
{
  "reviewer": {
    "reviewer_feedback": {
      "tags": [
        "San Jose Rental",
        "3 Bedroom House",
        "Pool Access"
      ],
      "summary": "Spacious 3 bedroom house near downtown San Jose with garage parking and pool access",
      "changed": false,
      "reason": "Proposal approved."
    }
  }
}
---NODE: Supervisor---
{
  "supervisor": {
    "turn_count": 5
  }
}
---GRAPH COMPLETE---
Temporary reviewer patch restored.

```

PART 4:

Pydantic Output Schema:

```
class PlannerOutput(BaseModel):
    tags: list[str] = Field(min_length=3, max_length=3)
    summary: str

    @field_validator("tags")
    @classmethod
    def validate_tag_lengths(cls, tags):
        for tag in tags:
            if len(tag) < 3 or len(tag) > 30:
                raise ValueError("Each tag must be 3 to 30 characters long")
        return tags

    @field_validator("summary")
    @classmethod
    def validate_summary_length(cls, summary):
        if len(summary.split()) > 25:
            raise ValueError("Summary must be no more than 25 words")
        return summary
```

Validation Retry + Loop Safety

```
def planner_node(state: AgentState) -> dict:
    print("---NODE: Planner---")
    attempts = state.get("planner_attempts", 0) + 1
    turn_ceiling = state.get("turn_ceiling", 10)

    result = model_client.complete(
        [{"role": "user", "content": prompt}],
        temperature=0.0,
        json_output=True,
    )

    try:
        proposal_data = json.loads(result["message"])
        proposal = PlannerOutput.model_validate(proposal_data)
    except (json.JSONDecodeError, ValidationError, TypeError, KeyError) as error:
        error_text = f"{type(error).__name__}: {error}"
        errors = state.get("validation_errors", []) + [error_text]
        status = "abandoned" if attempts >= turn_ceiling else "running"
```

```

return {
    "planner_proposal": {},
    "reviewer_feedback": {},
    "planner_attempts": attempts,
    "validation_error": error_text,
    "validation_errors": errors,
    "status": status,
}

return {
    "planner_proposal": proposal.model_dump(),
    "reviewer_feedback": {},
    "planner_attempts": attempts,
    "validation_error": "",
    "status": "running",
}

```

Validation back to planner:

```

if state.get("validation_error"):
    prompt += f"""\n

```

Previous output failed validation:
{state["validation_error"]}

Please correct the output and return valid JSON.
"""

Ceiling routing:

```

def router_logic(state: AgentState):
    if state.get("status") in {"completed", "abandoned"}:
        return END

    if not state["planner_proposal"]:
        if state.get("planner_attempts", 0) >= state.get("turn_ceiling", 10):
            return END
        return "planner"

    if not state["reviewer_feedback"]:
        return "reviewer"

```

```

if state["reviewer_feedback"].get("changed", False):
    if state.get("planner_attempts", 0) >= state.get("turn_ceiling", 10):
        return END
    return "planner"

```

```

return END

```

```

$ python3.11 validation_ceiling_smoke_test
TURN CEILING: 2 PLANNER ATTEMPTS
---NODE: Supervisor---
UPDATE FROM SUPERVISOR:
{
  "turn_count": 1
}
---NODE: Planner---
---MOCK PLANNER OUTPUT: invalid tag count and tag length---
UPDATE FROM PLANNER:
{
  "planner_proposal": {},
  "reviewer_feedback": {},
  "planner_attempts": 1,
  "validation_error": "ValidationError: 1 validation error for PlannerOutput\ntags\n List should have at least 3 items after validation, not 1 [type=too_short, input_value=['X'], input_type=list]\n For further information visit https://errors.pydantic.dev/2.13/v/too_short",
  "validation_errors": [
    "ValidationError: 1 validation error for PlannerOutput\ntags\n List should have at least 3 items after validation, not 1 [type=too_short, input_value=['X'], input_type=list]\n For further information visit https://errors.pydantic.dev/2.13/v/too_short"
  ],
  "status": "running"
}
---NODE: Supervisor---
UPDATE FROM SUPERVISOR:
{
  "turn_count": 2
}
---NODE: Planner---
---MOCK PLANNER OUTPUT: malformed JSON---
UPDATE FROM PLANNER:
{
  "planner_proposal": {},
  "reviewer_feedback": {},
  "planner_attempts": 2,
  "validation_error": "JSONDecodeError: Expecting value: line 1 column 1 (char 0)",
  "validation_errors": [
    "ValidationError: 1 validation error for PlannerOutput\ntags\n List should have at least 3 items after validation, not 1 [type=too_short, input_value=['X'], input_type=list]\n For further information visit https://errors.pydantic.dev/2.13/v/too_short",
    "JSONDecodeError: Expecting value: line 1 column 1 (char 0)"
  ],
  "status": "abandoned"
}
---NODE: Supervisor---
UPDATE FROM SUPERVISOR:
{
  "turn_count": 3
}
---FINAL RESULT---
planner_attempts: 2
turn_ceiling: 2
validation_error: JSONDecodeError: Expecting value: line 1 column 1 (char 0)
validation_errors stored: 2
status: abandoned
graph stopped: True
temporary mock restored: True

```

30-Run Schema Experiment:

Outcome	Count	Mean latency (ms)
Valid first attempt	30	2848.47
Valid after 1 retry	0	N/A
Valid after 2+ retries	0	N/A
Hit turn ceiling	0	N/A

Ceiling Comparison:

Ceiling	Completed	Completion Rate	Mean Latency
2	20/20	100%	2793.19 ms
10	20/20	100%	3916.19 ms

```
# HW2 Metrics

## Schema Validation - 30 Runs

| Outcome | Count | Mean latency (ms) |
|---|---|---|
| Valid first attempt | 30 | 2848.47 |
| Valid after 1 retry | 0 | N/A |
| Valid after 2+ retries | 0 | N/A |
| Hit turn ceiling | 0 | N/A |

## Turn Ceiling Comparison

| Ceiling | Runs | Completion Rate | Mean Latency (ms) |
|---|---|---|---|
| 2 | 20 | 100.0% | 2793.19 |
| 10 | 20 | 100.0% | 3916.19 |

Ceiling 2 is selected for deployment because completion rates matched and it had lower mean
latency.

## Adversarial Test

- Total runs: 5
- Ceiling used: 2
- Runs that hit the ceiling: 0
- Observed rate: 0.0%
- Why difficult: The rental text contains formatting instructions that conflict with the
required schema.
- Proposed fix: Clearly separate untrusted listing content from the model's output
instructions.
```

Deployment Choice: Both ceilings achieved a 100% completion rate. I selected a turn ceiling of 2 because it provides a tighter safeguard against unnecessary retries and had a lower observed mean latency of 2793.19 ms compared with 3916.19 ms for ceiling **10**.

Adversarial Test:

I used a rental listing containing conflicting instructions that asked the model to return exactly one tag named X and a summary of at least 40 words, which conflicts with the required Planner schema.

Turn ceiling: 2

Runs: 5

Hit turn ceiling: 0/5

Observed ceiling-hit rate: 0%

The adversarial input was designed to test whether instructions embedded in the rental content could override the required output format. In all five runs, the model ignored the conflicting content and produced valid output on the first attempt. A possible improvement is to clearly separate or delimit user-provided listing content as untrusted data so embedded instructions are less likely to affect model behavior.

Git Link: